

Bachelor in Computer Engineering – Management and Information Systems

Introduction to Cybersecurity

## **Lab 4**

### **Cryptography & Steganography CTF**

#### **Student Assignment**

|                   |                               |
|-------------------|-------------------------------|
| <b>Course</b>     | Introduction to Cybersecurity |
| <b>Lab</b>        | 4 of 4                        |
| <b>Topic</b>      | Cryptography & Steganography  |
| <b>Duration</b>   | 180–300 minutes               |
| <b>Difficulty</b> | Hard                          |
| <b>Flags</b>      | 14 (A1–A5, B1–B5, C1–C4)      |

## Learning Objectives

---

After completing this lab you will be able to:

- Apply classical cryptanalysis techniques (frequency analysis, Kasiski examination, crib-dragging)
- Understand and exploit AES-ECB mode weaknesses using a byte-at-a-time oracle attack
- Exploit an RSA small-exponent vulnerability ( $e = 3$ , cube root attack)
- Extract hidden data from image EXIF metadata and raw binary structures
- Perform LSB steganography extraction from RGB and alpha channels
- Crack steghide-protected JPEG payloads
- Recognise and exploit ECDSA nonce reuse to recover a private key
- Chain CTF challenges across multiple servers using derived credentials

## Scenario

---

**NovaCorp** has been breached. As a member of the incident response team, you have recovered a set of intercepted communications and suspicious image files from the attacker's exfiltrated archive.

Your mission is to break the ciphers, extract hidden data from the images, and reconstruct the attacker's communication channel step by step.

The investigation is divided into three modules hosted on separate servers:

- **Module A – Cryptography** (ServerA, 5 challenges): from simple substitution ciphers to asymmetric key attacks.
- **Module B – Steganography** (ServerB, 5 challenges): from EXIF metadata to LSB encoding and password-protected steganography.
- **Module C – Advanced Mix** (ServerC, 4 challenges): multilayer techniques combining cryptography and steganography. **Access to ServerC is gated** – you need two specific flags from Modules A and B first.

### Rules of engagement

- Only attack nodes within the lab environment
- Do not modify the VyOS or pfSense infrastructure configuration
- Document every step – commands run, scripts written, output obtained
- The flag format is H4U{ . . . } – capture the exact string
- If the lab breaks, notify your instructor

## The Gate Mechanism

### Important – read before starting

Access to **ServerC** is locked. The password is derived from two flags:

1. Flag **A5** – obtained from the RSA challenge on ServerA
2. Flag **B5** – obtained from the steghide JPEG on ServerB

Derive the ServerC password using the following formula:

$$\text{password} = \text{md5}(\text{flag\_A5} \parallel \text{flag\_B5})[:12]$$

Compute the MD5 hash of the concatenated flag strings and take the first 12 hexadecimal characters. This is the devops account password on ServerC.

## Network Topology

```
Homelab LAN (192.168.0.0/24)
|
[pfSense-LAB4] WAN: 192.168.0.18
|               LAN: 192.168.1.1/24
[VyOS-LAB4]
  eth0: 192.168.1.2/24 -- uplink to pfSense
  eth1: 10.10.1.1/24   -- Zone-A (ServerA)
  eth2: 10.10.2.1/24   -- Zone-B (ServerB)
  eth3: 10.10.3.1/24   -- Zone-C (ServerC) [GATED]
|
[ServerA] [ServerB] [ServerC]
10.10.1.10 10.10.2.10 10.10.3.10
Crypto    Stego      Advanced Mix
```

| Node    | IP         | Role                                | Entry                 |
|---------|------------|-------------------------------------|-----------------------|
| ServerA | 10.10.1.10 | Cryptography module (5 challenges)  | ssh admin@10.10.1.10  |
| ServerB | 10.10.2.10 | Steganography module (5 challenges) | ssh sysop@10.10.2.10  |
| ServerC | 10.10.3.10 | Advanced mix (4 challenges, gated)  | ssh devops@10.10.3.10 |

Table 1: Lab nodes overview

## Module A – Cryptography (ServerA)

**Entry:** `ssh admin@10.10.1.10` Password: N3xaTech!

All challenge files are in `/home/admin/mensajes/`.

### Challenge A1 – Classic Substitution

The file `A1_mensaje_interceptado.txt` contains an intercepted message. The text looks like English but every letter has been systematically shifted.

1. Read the file and observe the structure
2. Identify the cipher used (hint: the shift is exactly 13 positions)
3. Decode the message and extract the flag

*Deliverable:* the flag string in format `H4U{...}`.

### Challenge A2 – Block Cipher Oracle

The file `A2_admin_cipher.b64` contains a ciphertext encrypted with AES. The script `A2_oracle.py` provides an encryption oracle you can query.

1. Run `python3 A2_oracle.py` and study the interface
2. Understand why AES-ECB mode is vulnerable to chosen-plaintext attacks
3. Craft queries to recover the hidden message byte by byte
4. Extract the flag

*Deliverable: the flag string and a brief explanation of the ECB weakness.*

### **Challenge A3 – Polyalphabetic Cipher**

The file `A3_documento_clasificado.txt` is encrypted with a polyalphabetic substitution cipher. The key is a word, not a number.

1. Search for repeated trigrams in the ciphertext
2. Use the distances between repetitions to estimate the key length (Kasiski examination)
3. Once you know the key length, treat each column as a Caesar cipher
4. Recover the key and decrypt the full message

*Deliverable: the key word and the flag string.*

### **Challenge A4 – XOR Repeating Key**

The file `A4_mensaje_xor.hex` is a hex-encoded XOR-encrypted message. The key is shorter than the plaintext and repeats.

1. Decode the hex string to bytes
2. Estimate the key length using the index of coincidence or by trying lengths 1–20
3. Apply crib-dragging: XOR a known word against the ciphertext at every position
4. Recover the repeating key and decrypt the full message

*Deliverable: the key and the flag string.*

### Challenge A5 – RSA Weak Exponent (gate key)

The file `A5_rsa_challenge.txt` contains an RSA ciphertext and the public key parameters  $(n, e)$ . The public exponent is unusually small.

1. Read the ciphertext  $c$  and the parameters  $n$  and  $e$
2. Check whether  $m^e < n$  (i.e., no modular reduction occurred during encryption)
3. If so, compute the integer  $e$ -th root of  $c$  to recover  $m$
4. Convert  $m$  from integer to bytes and extract the flag

*Deliverable: the flag string. This flag is required to access ServerC.*

## Module B – Steganography (ServerB)

---

**Entry:** `ssh sysop@10.10.2.10` Password: `S3cure24!`

All challenge files are in `/srv/public/uploads/`.

### Challenge B1 – Metadata

The file `B1_oficina_novacorp.png` is a photograph of a Nova-Corp office.

1. Run `exiftool B1_oficina_novacorp.png`
2. Examine all metadata fields
3. Locate the field that contains the hidden flag

*Deliverable: the flag string.*

### Challenge B2 – Binary Structure

The file `B2_network_diagram.png` appears to be a valid PNG. However, something follows the official end of the file.

1. Find the position of the PNG IEND chunk marker (`49 45 4E 44`)
2. Read all bytes that appear after that position
3. Interpret those bytes as ASCII text

*Deliverable: the flag string.*

### **Challenge B3 – LSB in Red Channel**

The file `B3_documento_escaneado.png` hides data in the least-significant bit of the red channel of each pixel.

1. Open the image with Pillow (`from PIL import Image`)
2. Iterate over all pixels in row-major order
3. Extract the LSB of the red value of each pixel
4. Assemble the bits into bytes (8 bits per byte, MSB first)
5. Convert the bytes to ASCII until you reach the end of the flag

*Deliverable: the flag string.*

### **Challenge B4 – LSB in Alpha Channel**

The file `B4_novacorp_Logo.png` is an RGBA image. Data is hidden in the least-significant bit of the alpha channel.

1. Open the image and select channel A (alpha)
2. Apply the same LSB extraction technique as in B3 to the alpha values

*Deliverable: the flag string.*

### **Challenge B5 – Steghide JPEG (gate key)**

The file `B5_camara_seguridad.jpg` contains a hidden payload embedded with steghide. The passphrase is related to the company name in the scenario.

1. Run `steghide extract -sf B5_camara_seguridad.jpg`
2. When prompted for the passphrase, try words related to “Nova-Corp”
3. The extracted file contains the flag

*Deliverable: the flag string. This flag is required to access ServerC.*

## **Module C – Advanced Mix (ServerC)**

---

Derive the devops password from flags A5 and B5 before attempting this module. See Section 3 (The Gate Mechanism).

**Entry:** `ssh devops@10.10.3.10` Password: *derived*

All challenge files are in `/home/devops/retos/`.

### Challenge C1 – Multilayer

The file `C1_backup_tapes.jpg` requires three sequential steps to reveal the flag.

1. Run `exiftool C1_backup_tapes.jpg` and look for a hidden clue
2. Use the clue as a `steghide` passphrase to extract an embedded archive
3. Open the extracted archive and read the flag inside

*Deliverable: the flag string.*

### Challenge C2 – Onion Encoding

The file `C2_backup_Log.b64` is encoded in multiple nested layers.

1. Decode the file from base64
2. Decompress the result with `gzip`
3. Decode the output from base64 again
4. Decompress with `bzip2`
5. Read the plaintext flag

*Deliverable: the flag string.*

### Challenge C3 – Hash Length Extension

The file `C3_api_token.txt` contains a signed API token. The signature is computed as `SHA1(secret + message)`. The verification script `C3_verificador.py` accepts a forged token if the signature is valid for the extended message.

1. Read the original message and its MAC from `C3_api_token.txt`

2. Use the `h1extend` library to forge a new signature for the message extended with `&access=admin`
3. Submit the forged token to the verifier to obtain the flag

*Deliverable: the flag string and the forged message bytes.*

### Challenge C4 – ECDSA Nonce Reuse (Master)

The file `C4_ecdsa_signatures.txt` contains a set of ECDSA signatures over the `secp256k1` curve. The same nonce  $k$  was reused across two different messages.

1. Examine the file and identify two signatures that share the same  $r$  value
2. Recover the nonce  $k$  algebraically from the two  $(r, s, h)$  triples
3. Use  $k$  to recover the private key  $d$
4. Sign the challenge message with  $d$  and submit the signature to `C4_verificador.py`

*Deliverable: the flag string and the recovered private key.*

## Deliverables

---

Submit a technical report (3–5 pages) covering:

- **All 14 flags captured:** paste the exact flag strings for each challenge
- **Method summary per challenge:** one paragraph per flag explaining what vulnerability was exploited and how
- **Gate derivation:** show the computation used to derive the ServerC password from flags A5 and B5
- **Code snippets:** include any Python scripts you wrote
- **Reflection:** which challenge did you find hardest and why?

## Hints

---

**Stuck?** Hints are ordered by how much they reveal.

**Module A:**

1. A1: The cipher shifts each letter by 13 positions. `codecs.decode(text,`

`'rot_13')` works in one line.

2. A2: ECB encrypts each 16-byte block independently. Control the input so that only one unknown byte sits in your block.
3. A3: Find repeated sequences of 3+ letters. Measure the distances. The GCD of the distances is likely the key length.
4. A4: Guess a common English word (e.g. "the") and XOR it against the ciphertext at every offset. A correct placement reveals key bytes.
5. A5: `gmpy2.iroot(c, 3)` returns `(m, exact)`. If `exact` is `True`, you have the message.

#### Module B:

1. B1: `exiftool` shows all EXIF fields. Look at the `Artist` field.
2. B2: Search for the bytes `49 45 4E 44 AE 42 60 82` (IEND + CRC). Anything after that is appended data.
3. B3: `img.getpixel((x,y))[0] & 1` gives the LSB of red.
4. B4: `img.split()[3]` gives the alpha channel as a band.
5. B5: The passphrase is the company name in lowercase.

#### Module C:

1. C1: Check the `Comment` EXIF field for the steghide passphrase.
2. C2: Work layer by layer. `base64.b64decode` then `gzip.decompress` then `base64.b64decode` then `bz2.decompress`.
3. C3: `import hlexend; sha = hlexend.new('sha1')`. The secret length is 16 bytes.
4. C4: Two signatures with the same  $r$  value share the same  $k$ .  
$$k = (h_1 - h_2) \cdot (s_1 - s_2)^{-1} \pmod{n}.$$

## Tools Reference

The following tools are available on the servers:

| Tool         | Available on     | Use                                |
|--------------|------------------|------------------------------------|
| python3      | All servers      | Scripting, decoding, cryptanalysis |
| pycryptodome | ServerA          | AES, RSA primitives                |
| gmpy2        | ServerA          | Integer cube root for RSA attack   |
| exiftool     | ServerB, ServerC | Read/write image EXIF metadata     |
| steghide     | ServerB, ServerC | Embed/extract steghide payloads    |
| python3-pil  | ServerB, ServerC | Image pixel manipulation           |
| hlexend      | ServerC          | SHA-1 length extension attack      |

Table 2: Tools available on each server